Conclusion clé — Comment résoudre (presque) toutes les questions du sujet

1) Checklist universelle avant d'écrire

1. **Identifier l'objet** : mot (A*), liste, arbre, paire (ℕ×ℕ), langage (Dyck), relation (préfixe/lexico), etc.
2. **Choisir le bon type d'induction** :
   - *Sur les entiers* : induction simple/forte.
   - *Sur des objets construits* (mots, listes, arbres) : **induction structurelle**.
   - *Quand on diminue une mesure* (longueur, hauteur, ordre bien fondé) : **induction bien fondée**.
3. **Écrire la propriété P(·)** précisément (avec notations exactes) *avant* la preuve.
4. **Dérouler Base → Induction** en suivant la définition de l'objet (cas du constructeur).
5. **Conclure** proprement (phrase finale = points faciles).

2) Patrons prêts à copier

```
• Induction simple (n ∈ ℕ)
Base : montrer P(0) (ou P(1)).
H.I. : supposer P(n) vraie.
Pas : démontrer P(n+1) à partir de P(n). Conclure.
```

• Induction forte (n ∈ ℕ) Base : montrer P(0). H.I. : supposer P(k) vraie pour tous k ≤ n. Pas : démontrer P(n+1) en utilisant $\{P(k)\}_{k\leq n}$. Conclure.

• Induction structurelle (mots/listes/arbres) Base(s) : cas le(s) plus simple(s) (ε, nil, feuille/∅). Pas : supposer la propriété vraie pour les sous-objets (u, x, g/d), prouver pour l'objet construit (u·a, n|x, ⟨g,a,d⟩). Conclure.

• Induction bien fondée (mesure m < m(x)) Base : cas minimal(s). Pas : supposer P(y) pour tous y < x (selon la mesure), démontrer P(x). Conclure.

Recettes rapides par type d'exercice

A) Mots (A*), longueurs et miroirs

- **But fréquent** : |uv|=|u|+|v|, $|w|=\sum|w|_a$, ~ (~w)=w, |~w|=|w|.
- **Méthode** : induction structurelle sur le 2e argument (ou sur w).
  Base ε. Pas u·a : réécrire la propriété en utilisant la définition (ajout d'une lettre).
- **Astuce** : quand on colle une lettre, la longueur augmente de 1 et le compteur de cette lettre aussi.

B) Langage de Dyck (équilibres a/b)

- **But fréquent** : montrer l'équivalence "définition inductive" ⇔ "comptage + condition de préfixes".
- **Direction inductive** (E1→E2) : induction structurelle sur w = a·u·b·v. Cas des préfixes : (avant a) / (entre a et b) / (après b) + I.H. sur u et v.

- **Direction inverse** (E2→E1) : induction bien fondée sur |w|. w≠ε ⇒ w commence par a et admet sa *première* fermeture b : écrire w = a·u·b·v, appliquer I.H. à u et v, reconstruire.
- **Astuce** : pense “pile” (a = push, b = pop). Jamais de dette de b dans un préfixe.

C) Ordres sur A* : préfixe, lexicographique, bien-fondé

- **Préfixe** : u ≤ v ⇔ ∃w, v = u·w. Partiel, bien fondé. Pour montrer l’équivalence avec la version “inductive”, fais une double inclusion par induction.
- **Lexicographique** (A total) : total mais en général pas bien fondé. Contre-exemple : b > ab > aab > aaab > …
- **Astuce** : si l’énoncé parle de “minimal”, vérifie d’abord le bien-fondé.

D) Listes (nil, n|x), somme s, longueur ℓ, renversement rev

- **But fréquent** : s(x|y)=s(x)+s(y), ℓ(x|y)=ℓ(x)+ℓ(y), ℓ(rev x)=ℓ(x), rev(rev x)=x.
- **Méthode** : induction structurelle sur la 1re liste ; base nil ; pas n|x.
- **Astuce** : `rev(n|x) = rev(x) || n` (ajout en fin). Garde la définition sous les yeux.

E) Arbres binaires : h, n, f ; arbres stricts

- **Formules** :
  - h(∅)=0, h(•)=1, h(⟨g,a,d⟩)=1+max(h(g),h(d))
  - n(∅)=0, n(•)=1, n(⟨g,a,d⟩)=1+n(g)+n(d)
  - f(∅)=0, f(•)=1, f(⟨g,a,d⟩)=f(g)+f(d)
- **Bornes** : n(t) ≤ 2^{h(t)} − 1, f(t) ≤ 2^{h(t)-1}. Induction structurelle : majore à gauche et droite, puis utilise H = 1 + max(…)
- **Arbre strict** : n = ar + 1 et n = 2f − 1. Induction structurelle : feuille (OK), puis ⟨g,a,d⟩ avec I.H. sur g et d.
- **Astuce** : teste tes bornes sur les deux extrêmes — *chaîne* (très peu de nœuds) et *plein* (atteint la borne).

F) Relations inductives sur ℕ×ℕ (Inf1 / Inf2), ensemble D des multiples

- **Idée** : ces règles *encodent* souvent ≤ ou la divisibilité. Prouve la propriété caractéristique (⇔) par double implication.
- **Inf1 / Inf2** : montrer “appartient” ⇔ “$n_1 \le n_2$” avec un sens “règles ⇒ ≤” (lecture) + “≤ ⇒ règles” (construction par pas).
- **Multiples D** : (n,0) base ; (n,n′) ⇒ (n,n+n′). Montrer (n,n′) ∈ D ⇔ ∃k, n′ = k·n par induction sur k (ou structurelle).
- **Astuce** : *dessine* quelques premiers couples sur une grille, tu vois le motif.

Pièges fréquents & comment les éviter

- **Oublier la base** : toujours traiter ε / nil / ∅ / feuille.
- **Induction mal calée** : suis exactement le constructeur de l’objet (u·a / n|x / ⟨g,a,d⟩).
- **Confondre ordre préfixe et lexico** : préfixe = partiel & bien fondé ; lexico = total & pas bien fondé (en général).
- **Dyck** : vérifier tous les préfixes (pas seulement la fin). La première fermeture b est la clé de la décomposition.
- **Arbres** : h = 1 + max(…) (pas somme), f = f(g)+f(d) (pas +1), n = 1 + n(g)+n(d).

- **Classes d'équivalence vs ordre** : équivalence = réflexive + symétrique + transitive ; ordre = réflexive + antisymétrique + transitive.

Auto-contrôles en 20 secondes

- **Ai-je déclaré clairement P(·)** juste avant la preuve ?
- **Base(s) écrite(s)** ? (ε / nil / ∅ / feuille)
- **Hypothèse(s) d'induction** bien posées (j'indique sur quoi porte l'I.H.) ?
- **Pas d'induction** : j'applique mécaniquement la définition du constructeur.
- **Conclusion** : phrase finale explicite ("Par (base) et (induction), P(·) est vraie pour tout …").

Gabarits de réponse (à coller / adapter)

1) Induction structurelle (mots)

```
On prouve la propriété P(w) par induction structurelle sur w ∈ A*.
Base : w = ε. (Détailler et conclure P(ε).)
Pas : supposer P(u) vraie et montrer P(u·a).
      (Utiliser la définition de la propriété et la construction u·a.)
Conclusion : par induction, P(w) vraie pour tout w ∈ A*.
```

2) Induction structurelle (arbres)

```
On prouve P(t) par induction structurelle.
Bases : t = ∅ et t = • (vérifications directes).
Pas : t = ⟨g,a,d⟩. Supposer P(g) et P(d) vraies (I.H.), déduire P(t) par
les définitions (h, n, f, ...).
Conclusion.
```

3) Dyck (sens "préfixes" → "inductif")

```
Induction bien fondée sur |w|.
Si w = ε, c'est ok. Sinon w commence par a. Soit b la première fermeture
correspondante.
On écrit w = a u b v avec u, v plus courts.
Par I.H., u, v appartiennent au langage inductif.
Par règle (I), a u b v appartient. Conclusion.
```

4) Égalité de deux définitions (ex. préfixe)

```
Montrer D1 = D2 par deux inclusions.
(i) D2 ⇒ D1 : induction sur la dérivation de D2.
(ii) D1 ⇒ D2 : récurrence (souvent sur la longueur). Conclusion.
```

À retenir en une minute

- **Choisis la bonne induction** en lisant la définition de l'objet.
- **Écris P(·)** proprement, puis déroule Base → Pas d'induction → Conclusion.
- **Pour Dyck** : "première fermeture" + pile ; pour **préfixe** : deux définitions équivalentes ; pour **lexico** : total mais pas bien fondé ; pour **arbres** : h, n, f bien définies ; pour **listes** : s, ℓ, rev suivent la structure.
- **Finis par une phrase de conclusion** (ça rapporte).

Avec ces recettes, tu peux attaquer sereinement les variantes possibles de l'exam dans l'heure. Bonne réussite ! 💪

Conclusion — Induction sur ℕ & Induction structurelle (version simple et claire)

1) Idées clés à retenir

- **Induction simple sur ℕ** : on prouve d'abord la *base* (souvent n=0 ou n=1), puis l'*hérédité* : P(n) ⇒ P(n+1). Si ces deux étapes sont vraies, alors P(n) est vraie pour tout n.
- **Induction forte** : pour prouver P(n), on peut supposer vraies *toutes* les P(k) pour k < n. Utile quand la preuve pour n dépend de plusieurs valeurs plus petites (ex : suites définies par plusieurs cas).
- **Induction structurelle** : pour des objets construits par *règles* (mots, arbres), on prouve P sur *toutes les bases*, puis on montre que si P est vraie sur les sous-objets, alors P est vraie sur l'objet construit avec eux.

2) Notations utilisées

- **Mots sur A = {a,b}** : A* est l'ensemble de tous les mots finis. $|w|_a$ = nombre de « a » dans w ; $|w|_b$ = nombre de « b » dans w. Un *préfixe* u de v signifie qu'il existe x tel que u·x = v. Un *préfixe propre* est un préfixe différent du mot entier.
- **Arbres binaires** :
  - Base : ∅ (arbre vide) est un arbre.
  - Construction : si g et d sont des arbres et a est une étiquette, alors (a, g, d) est un arbre.

  Définitions usuelles (par récurrence structurelle) :
  - Hauteur h : h(∅)=0 ; h((a,g,d)) = 1 + max(h(g), h(d)).
  - Nombre de nœuds n : n(∅)=0 ; n((a,g,d)) = 1 + n(g) + n(d).

- Nombre de feuilles f : f(∅)=0 ; f((a,∅,∅))=1 ; sinon f((a,g,d)) = f(g)+f(d).
- Arêtes ar : ar(∅)=0 ; ar((a,g,d)) = ar(g)+ar(d) + [g≠∅] + [d≠∅].

Arbre binaire strict : non vide et chaque nœud a soit 0 enfant (feuille) soit 2 enfants (jamais 1 seul).

3) Modèle de rédaction — Induction structurelle

1. **Rappeler la définition inductive** (bases et règles de construction).
2. **Énoncer clairement la propriété P(x)** à montrer pour tout objet x.
3. **Cas de base** : prouver P pour chaque base.
4. **Étape inductive** : supposer P vraie pour les sous-objets (hypothèses d'induction), puis prouver P pour l'objet construit.
5. **Conclusion** : « donc, par induction structurelle, P est vraie pour tous les objets ».

---

4) Exemple guidé (mots) — Langage L sur A={a,b}

Définition de L :

```
(B) b ∈ L
(I) si u ∈ L et v ∈ L alors a·u·v ∈ L
```

À prouver :

1. (1) Pour tout w∈L : $|w|_b = |w|_a + 1$.
2. (2) Pour tout w∈L et tout préfixe propre w′ de w : $|w'|_a \geq |w'|_b$.

Solution pas à pas (simple et complète)

(1) Égalité globale sur chaque mot de L

- *Base* : w=b. Alors $|w|_a=0$ et $|w|_b=1$. Donc $|w|_b = |w|_a + 1$ (vrai).
- *Induction* : supposons vrais pour u,v∈L :
  $|u|_b = |u|_a + 1$ et $|v|_b = |v|_a + 1$.
  Pour w = a·u·v :
  $|w|_a = 1 + |u|_a + |v|_a$
  $|w|_b = |u|_b + |v|_b = (|u|_a+1) + (|v|_a+1) = (1+|u|_a+|v|_a) + 1 = |w|_a + 1$.
  Donc l'égalité est vraie pour a·u·v.

(2) Sécurité des préfixes : tout préfixe propre a au moins autant de « a » que de « b ».

On prouve P(w) : « pour tout préfixe propre w′ de w, $|w'|_a \geq |w'|_b$ » par induction structurelle.

- *Base* : w=b. Le seul préfixe propre est ε : $|\varepsilon|_a=|\varepsilon|_b=0 \Rightarrow$ OK.
- *Induction* : w=a·u·v. Supposons P(u) et P(v) vraies. Prenons un préfixe propre w′ de w. Quatre cas simples :
  a. w′=ε ⇒ 0≥0, OK.
  b. w′=a ⇒ 1≥0, OK.
  c. w′=a·x avec x préfixe propre de u.
  Par P(u), $|x|_a \geq |x|_b$. Donc $|w'|_a=1+|x|_a \geq 1+|x|_b=|w'|_b$.
  d. w′=a·u·y avec y préfixe propre de v.
  Par (1) sur u : $|u|_b=|u|_a+1$. Par P(v) sur y : $|y|_a \geq |y|_b$.
  $|w'|_a = 1 + |u|_a + |y|_a$
  $|w'|_b = |u|_b + |y|_b = (|u|_a+1) + |y|_b$
  Donc $|w'|_a - |w'|_b = (|y|_a - |y|_b) \geq 0$.
  Tous les cas sont bons ⇒ P(w) est vraie.

Petit exemple concret pour « voir » :
Prenons u=b et v=b. Alors w_1=a·b·b. Compte : $|w_1|_a=1$, $|w_1|_b=2$ (OK pour (1)).
Reprenons u=w_1 et v=b : w_2=a·w_1·b = a a b b b. Compte : $|w_2|_a=2$, $|w_2|_b=3$. Tous les préfixes vérifient (2).

---

5) Exemple guidé (arbres) — Deux formules classiques

1. **Borne sur les nœuds** : pour tout arbre t, `n(t) ≤ 2^{h(t)} − 1`.
2. **Arbres stricts** :
   - (a) `n(t) = ar(t) + 1`
   - (b) `n(t) = 2·f(t) − 1`

Preuves courtes et claires

(1) n(t) ≤ 2^{h(t)} − 1

- *Base* : t=∅. n=0, h=0 ⇒ 0 ≤ 1−1 = 0. OK.
- *Induction* : t=(a,g,d). Par hypothèse d'induction : n(g) ≤ 2^{h(g)}−1 et n(d) ≤ 2^{h(d)}−1. Posons M=max(h(g),h(d)), alors h(t)=M+1.
  n(t)=1+n(g)+n(d) ≤ 1+(2^{h(g)}−1)+(2^{h(d)}−1) ≤ 2·2^M−1 = 2^{M+1}−1 = 2^{h(t)}−1.
  Donc l'inégalité est vraie.

(2) Arbres stricts

(a) n(t) = ar(t) + 1

- *Base (feuille)* : n=1, ar=0 ⇒ 1=0+1. OK.
- *Induction* : t=(a,g,d) avec g≠∅ et d≠∅ (strict). Alors ar(t)=ar(g)+ar(d)+2. Par hypothèse d'induction : n(g)=ar(g)+1 et n(d)=ar(d)+1.

n(t)=1+n(g)+n(d)=1+(ar(g)+1)+(ar(d)+1)=ar(g)+ar(d)+3=(ar(g)+ar(d)+2)+1=ar(t)+1.
C'est prouvé.

(b) n(t) = 2·f(t) − 1

- *Base (feuille)* : n=1, f=1 ⇒ 1 = 2·1 − 1. OK.
- *Induction* : t=(a,g,d) strict interne, donc g,d≠∅. Par hypothèse : n(g)=2f(g)−1 et n(d)=2f(d)−1 ; et f(t)=f(g)+f(d).
  n(t)=1+n(g)+n(d)=1+(2f(g)−1)+(2f(d)−1)=2(f(g)+f(d))−1=2f(t)−1.
  C'est prouvé.

---

6) Mini-fiche « comment rédiger vite et propre »

- **Étape 1** : écrire clairement la définition par cas (bases + règle(s)).
- **Étape 2** : énoncer la propriété P avec les bonnes notations ($|w|_a$, préfixe, h, n, f, ar…).
- **Étape 3** : traiter *tous* les cas de base.
- **Étape 4** : pour l'étape inductive, analyser *toutes* les formes possibles de l'objet construit (par ex. pour les préfixes de a·u·v : ε, a, a·(préfixe de u), a·u·(préfixe de v)).
- **Étape 5** : utiliser l'hypothèse d'induction uniquement sur les sous-objets autorisés par la définition.
- **Étape 6** : conclure explicitement par « induction » ou « induction structurelle ».

7) Exemple rapide (induction simple sur ℕ)

Montrer : pour tout n≥1, 1+2+…+n = n(n+1)/2.

Preuve courte

- *Base* n=1 : 1 = 1·2/2 ⇒ vrai.
- *Hérédité* : supposons vrai pour n. Alors pour n+1 :
  1+…+n+(n+1) = [n(n+1)/2] + (n+1) = (n+1)(n/2 + 1) = (n+1)(n+2)/2.
  Donc la formule est vraie pour n+1. Conclusion par induction.

---

8) Erreurs fréquentes (à éviter)

- Oublier un cas de base.
- Utiliser l'hypothèse d'induction sur un objet qui n'est pas un « sous-objet » de la construction.
- Mélanger définitions (ex : confondre préfixe et sous-mot quelconque).
- Ne pas traiter toutes les formes possibles d'un préfixe ou d'un nœud.

9) Exercice d'entraînement (à faire seul, plan fourni)

Ordre préfixe : On définit « $u \leq_{pref} v$ » par :
(B) $\varepsilon \leq v$; (I) si $u \leq v$ alors $a \cdot u \leq a \cdot v$ (pour toute lettre a).
Montrer que « $u \leq_{pref} v$ » est équivalent à « il existe w tel que $v = u \cdot w$ ».

Plan très simple

- *Sens 1* (règles ⇒ existence de w) : induction structurelle sur la dérivation de « $u \leq v$ ». Traiter le cas base ε, puis le cas $a \cdot u \leq a \cdot v$.
- *Sens 2* (existence de w ⇒ règles) : induction sur la longueur de w. Cas $w=\varepsilon$ puis $w=a \cdot x$.

---

10) Résumé express

- Induction simple : base + hérédité $n \to n+1$.
- Induction forte : on suppose vrai pour tous les $k<n$.
- Induction structurelle : bases + « si vrai sur les morceaux, alors vrai sur l'assemblage ».
- Mots L : la règle $a \cdot u \cdot v$ ajoute 1 « a » et conserve l'écart « b = a + 1 » ; les préfixes restent sûrs ($|a| \geq |b|$).
- Arbres : toujours exprimer h, n, f, ar en fonction des sous-arbres et conclure par récurrence.

Si tu veux, je peux mettre tout ceci directement dans ton modèle exact (les mêmes) pour un TD prêt à imprimer, ou ajouter d'autres exemples détaillés (arbre strict, parcours préfixe, etc.).

Exemple unique — Arbre binaire : notations + « comment trouver l'induction » + preuve guidée

1) Les notations (très simple)

- **Définition inductive des arbres binaires**
  - Base : ∅ (arbre vide) est un arbre.
  - Construction : si `g` et `d` sont des arbres et `a` est une étiquette (lettre, nombre…), alors `(a, g, d)` est un arbre (a = racine, g = sous-arbre gauche, d = sous-arbre droit).
- **Fonctions usuelles (définies elles aussi par récurrence)**
  - Hauteur `h` : `h(∅)=0`, `h((a,g,d)) = 1 + max(h(g), h(d))`.
  - Nombre de nœuds `n` : `n(∅)=0`, `n((a,g,d)) = 1 + n(g) + n(d)`.
  - Nombre de feuilles `f` :
    - `f(∅)=0`, `f((a,∅,∅))=1` (une feuille),

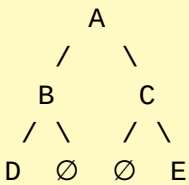
    - sinon `f((a,g,d)) = f(g) + f(d)`.
  - Nombre d'arêtes `ar` : `ar(∅)=0`, `ar((a,g,d)) = ar(g)+ar(d) + [g≠∅] + [d≠∅]` (on ajoute 1 par enfant non vide).
- **Arbre binaire strict** : non vide, et chaque nœud a soit 0 enfant (feuille), soit 2 enfants (jamais exactement 1).

2) Comment « trouver l'induction » (la bonne propriété P)

1. **Regarder la construction** : vos objets naissent soit de `∅`, soit d'un collage `(a,g,d)`. Donc votre preuve doit traiter exactement ces deux formes.
2. **Écrire la propriété P(t)** en fonction de ce que modifie la construction. Exemple : des inégalités comme `n(t) ≤ 2^{h(t)} − 1` ou des identités comme `n(t)=ar(t)+1` ou, en strict, `n(t)=2f(t)−1`.
3. **Cas de base** : prouver P(∅) (et P(feuille) si nécessaire).
4. **Étape inductive** : supposer P(g) et P(d) vraies (hypothèses d'induction), puis démontrer P((a,g,d)). On *exprime tout* (h, n, f, ar…) par les mêmes choses sur g et d — c'est la clé.

3) Un seul exemple complet

Arbre choisi :



Forme inductive : `t = (A, (B, (D, ∅, ∅), ∅), (C, ∅, (E, ∅, ∅)))`.

Note : cet arbre n'est **pas** strict (B et C ont un seul enfant).

Étape A — Calcul des notations sur l'exemple

- **Hauteurs**
  - h(D)=1, h(E)=1 (feuilles)
  - h(B)=1+max(h(D),0)=2 ; h(C)=1+max(0,h(E))=2
  - h(A)=1+max(h(B),h(C))=1+max(2,2)=3
- **Nombre de nœuds**
  - n(D)=1, n(E)=1
  - n(B)=1+n(D)+0=2 ; n(C)=1+0+n(E)=2

    - n(A)=1+n(B)+n(C)=1+2+2=5
- **Nombre de feuilles**
    - f(D)=1, f(E)=1
    - f(B)=f(D)+f(∅)=1+0=1 ; f(C)=f(∅)+f(E)=0+1=1
    - f(A)=f(B)+f(C)=1+1=2
- **Nombre d'arêtes**
    - ar(D)=0, ar(E)=0
    - ar(B)=ar(D)+0 + [D≠∅]+[∅≠∅] = 0+0+1+0=1
    - ar(C)=0+0 + 0+1 = 1
    - ar(A)=ar(B)+ar(C) + 1 + 1 = 1+1+2=4

Sanity check visuel : il y a 5 nœuds (A,B,C,D,E) et 4 arêtes (A–B, A–C, B–D, C–E). Tout colle.

4) Preuve guidée n°1 — « Pour tout arbre t, n(t) ≤ 2^{h(t)} − 1 »

Pourquoi cette P(t) ? Parce que quand on colle deux sous-arbres sous une racine, la hauteur prend un `max` et s'incrémente de 1, tandis que le nombre de nœuds s'additionne puis +1 : c'est parfait pour comparer avec une puissance de 2.

Rédaction (ultra claire)

- **Propriété** P(t) : `n(t) ≤ 2^{h(t)} − 1` pour tout arbre t.
- **Base** : t=∅. Alors n=0 et h=0, donc `0 ≤ 2^0 − 1 = 0`. OK.
- **Induction** : soit t=(a,g,d). On suppose P(g) et P(d) vraies : `n(g) ≤ 2^{h(g)} − 1` et `n(d) ≤ 2^{h(d)} − 1`.
  Posons M = max(h(g), h(d)). Alors h(t)=M+1.
  n(t) = 1 + n(g) + n(d) ≤ 1 + (2^{h(g)} − 1) + (2^{h(d)} − 1) ≤ 1 + (2^{M} − 1) + (2^{M} − 1) = 2·2^{M} − 1 = 2^{M+1} − 1 = 2^{h(t)} − 1.
  Conclusion : P(t) est vraie pour tous les arbres par induction structurelle.

Vérification sur notre exemple : h(t)=3 et n(t)=5, donc `5 ≤ 2^3−1 = 7`, correct.

5) Preuve guidée n°2 (bonus) — Arbres stricts : « n(t) = 2·f(t) − 1 »

Pourquoi cette P(t) ? En arbre strict, chaque nœud interne a 2 enfants ⇒ le « +1 » des nœuds se répartit exactement sur les feuilles. C'est une identité classique.

Rédaction courte

- **Propriété** P(t) : `n(t) = 2·f(t) − 1` pour tout arbre strict non vide.
- **Base** (feuille) : n=1, f=1 ⇒ `1 = 2·1 − 1`, OK.
- **Induction** : t=(a,g,d) strict interne, donc g≠∅ et d≠∅, et P(g), P(d) vraies : `n(g)=2f(g)−1`, `n(d)=2f(d)−1`, et `f(t)=f(g)+f(d)`.
  n(t) = 1 + n(g) + n(d) = 1 + (2f(g)−1) + (2f(d)−1) = 2(f(g)+f(d)) − 1 = 2f(t) − 1.
  Conclusion : identité vraie pour tout arbre binaire strict.

Attention : notre arbre d'exemple n'est pas strict, donc cette formule ne s'y applique pas (ici n=5, f=2 ⇒ 2f−1=3 ≠ 5).

Résumé pratique (recette que tu peux réutiliser)

1. **Recopier la définition inductive** (cas ∅ et cas (a,g,d)).
2. **Choisir la propriété P(t)** en observant comment h, n, f, ar se calculent à partir de g et d.
3. **Base** : prouver P(∅) (et P(feuille si besoin)).
4. **Induction** : supposer P(g) et P(d), puis exprimer tout sur (a,g,d) et conclure.
5. **Vérifier sur 1–2 petits arbres** pour « sentir » la propriété.

6) Exercice d'entraînement (facile, même méthode)

Montrer pour *tout* arbre (pas forcément strict) non vide : `ar(t) = n(t) − 1`.

Indications

- Base (feuille) : n=1 et ar=0 ⇒ OK.
- Induction : pour t=(a,g,d), écrire `n(t)=1+n(g)+n(d)` et `ar(t)=ar(g)+ar(d)+[g≠∅]+[d≠∅]`, puis utiliser les hypothèses d'induction sur g et d. Traiter les cas `g=∅` ou `d=∅` proprement (un seul enfant ⇒ on ajoute 1 seule arête).

Exemple unique — Arbre binaire (version aérée + lisible)

1) Notations (claires et courtes)

Arbres binaires — Définition inductive

- *Base* : ∅ est un arbre.
- *Construction* : si `g` et `d` sont des arbres et `a` est une étiquette, alors `(a, g, d)` est un arbre (racine `a`, sous-arbre gauche `g`, sous-arbre droit `d`).

Fonctions standards (définies aussi par récurrence)

- Hauteur : `h(∅)=0`, `h((a,g,d)) = 1 + max(h(g), h(d))`.
- Nombre de nœuds : `n(∅)=0`, `n((a,g,d)) = 1 + n(g) + n(d)`.
- Nombre de feuilles :

  - `f(∅)=0`, `f((a,∅,∅))=1` (une feuille),
  - sinon `f((a,g,d)) = f(g) + f(d)`.
- Nombre d'arêtes : `ar(∅)=0`, `ar((a,g,d)) = ar(g)+ar(d) + [g≠∅] + [d≠∅]` (on ajoute 1 par enfant non vide).

Arbre binaire strict : non vide, et chaque nœud a soit 0 enfant, soit 2 enfants (jamais exactement 1).

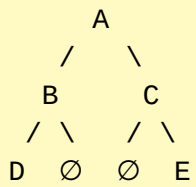
---

2) Choisir la « bonne » récurrence (recette rapide)

1. **Regarde la construction** : tous les objets sont soit `∅`, soit `(a,g,d)` ⇒ ta preuve aura exactement ces deux cas.
2. **Écris la propriété P(t)** en termes de `g` et `d`.
   Exemples utiles :
   - $n(t) \le 2^{h(t)} - 1$ (inégalité « puissance de 2 »),
   - $ar(t) = n(t) - 1$ (identité structurelle),
   - *(strict)* $n(t) = 2 \cdot f(t) - 1$ (lien nœuds ↔ feuilles).
3. **Base(s)** : prouve P(∅) (et P(feuille) si nécessaire).
4. **Étape** : suppose P(g) et P(d), exprime tout pour `(a,g,d)`, puis conclue.

---

3) Un arbre d'exemple (on calcule tout)



Forme inductive : `t = (A, (B, (D, ∅, ∅), ∅), (C, ∅, (E, ∅, ∅)))`.

Remarque : cet arbre n'est **pas** strict (B et C ont un seul enfant).

---

4) Calcul pas à pas des notations sur cet arbre

Hauteur

- Feuilles : `h(D)=1`, `h(E)=1`.
- `h(B)=1+max(h(D),0)=1+max(1,0)=2`.

- `h(C)=1+max(0,h(E))=1+max(0,1)=2`.
- `h(A)=1+max(h(B),h(C))=1+max(2,2)=3`.

Nombre de nœuds

- Feuilles : `n(D)=1`, `n(E)=1`.
- `n(B)=1+n(D)+0=2`, `n(C)=1+0+n(E)=2`.
- `n(A)=1+n(B)+n(C)=1+2+2=5`.

Nombre de feuilles

- Feuilles : `f(D)=1`, `f(E)=1`.
- `f(B)=f(D)+f(∅)=1+0=1`, `f(C)=f(∅)+f(E)=0+1=1`.
- `f(A)=f(B)+f(C)=1+1=2`.

Nombre d'arêtes

- Feuilles : `ar(D)=0`, `ar(E)=0`.
- `ar(B)=ar(D)+ar(∅)+[D≠∅]+[∅≠∅]=0+0+1+0=1`.
- `ar(C)=ar(∅)+ar(E)+[∅≠∅]+[E≠∅]=0+0+0+1=1`.
- `ar(A)=ar(B)+ar(C)+[B≠∅]+[C≠∅]=1+1+1+1=4`.

Vérification visuelle : 5 nœuds (A,B,C,D,E) et 4 arêtes (A–B, A–C, B–D, C–E) — cohérent.

---

5) Preuve guidée — $n(t) \le 2^{h(t)} - 1$ (tous arbres)

Idée : ajouter une racine au-dessus de `g` et `d` fait `h(t)=1+max(h(g),h(d))` et `n(t)=1+n(g)+n(d)`. Cette structure « colle » naturellement avec une borne en 2^h.

Propriété P(t) : $n(t) \le 2^{h(t)} - 1$.

Base : t=∅. `n=0`, `h=0` ⇒ $0 \le 2^0 - 1 = 0$.

Étape : t = (a,g,d).

- Hypothèses d'induction : $n(g) \le 2^{h(g)} - 1$ et $n(d) \le 2^{h(d)} - 1$.
- Posons $M = \max(h(g), h(d))$ ⇒ $h(t) = M + 1$.
- Calcul :

  $n(t) = 1 + n(g) + n(d)$

  $\le 1 + (2^{h(g)} - 1) + (2^{h(d)} - 1)$

$\leq 1 + (2^M - 1) + (2^M - 1)$
$= 2 \cdot 2^M - 1 = 2^{M+1} - 1 = 2^{h(t)} - 1$.

Conclusion : P(t) est vraie pour tous les arbres par récurrence structurelle.

Test sur notre arbre : $h(t)=3$, $n(t)=5$ ⇒ $5 \leq 2^3 - 1 = 7$ ✔

---

6) Bonus — Arbres stricts : $n(t) = 2 \cdot f(t) - 1$

Idée : chaque nœud interne strict a 2 enfants — l'« équilibre » nœuds/feuilles donne l'identité.

Base (feuille) : $n=1$, $f=1$ ⇒ $1 = 2 \cdot 1 - 1$.

Étape : t=(a,g,d) strict interne (donc $g \neq \emptyset$ et $d \neq \emptyset$).

- HI : $n(g)=2f(g)-1$, $n(d)=2f(d)-1$.
- Comme $f(t)=f(g)+f(d)$:
  $n(t)=1+n(g)+n(d)$
  $= 1 + (2f(g)-1) + (2f(d)-1)$
  $= 2(f(g)+f(d)) - 1 = 2f(t) - 1$.

Attention : notre arbre d'exemple n'est pas strict — cette formule ne s'y applique pas.

---

7) Exercice d'entraînement — $ar(t) = n(t) - 1$ (arbres non vides)

Indications très courtes

Base (feuille) : $n=1$, $ar=0$ ⇒ $ar = n - 1$.

Étape : t=(a,g,d).

- Écris $n(t)=1+n(g)+n(d)$.
- Écris $ar(t)=ar(g)+ar(d)+[g \neq \emptyset]+[d \neq \emptyset]$.
- Applique l'HI à g et d, puis discute les cas « un seul enfant » vs « deux enfants » pour bien retomber sur $ar = n - 1$.

---

8) Résumé-utilisateur (à répliquer à l'examen)

1. Je recopie la **définition inductive** (∅ et (a,g,d)).

2. Je choisis une **propriété P(t)** exprimée via `g` et `d` (ex. `n(t) ≤ 2^h(t) − 1`).
3. **Bases** : je prouve P(∅) (et P(feuille si besoin).
4. **Étape** : j'assume P(g), P(d), je traduis `n,h,f,ar` de `(a,g,d)` en fonction de `g,d`, j'aligne les calculs, je conclus.
5. Je **vérifie** sur un petit arbre (comme ci-dessus) pour un « sanity check » rapide.

Conclusion clé — Méthode unique pour 90% des questions

1. **Identifier l'objet** : entier, mot (A*), liste, arbre, relation inductive, langage (Dyck), ordre (préfixe/lexico)…
2. **Choisir la bonne induction** :
   - ℕ : simple (n→n+1) ou forte (tous k≤n).
   - *Objets construits* : **induction structurelle** (bases + constructeurs).
   - *Mesure qui diminue* (longueur, hauteur, etc.) : **induction bien fondée**.
3. **Écrire clairement la propriété P(·)** avant la preuve (avec les notations exactes).
4. **Dérouler** : Bases → Pas d'induction (en suivant la définition de l'objet).
5. **Conclure** : une phrase explicite ("Donc, par induction, …").

Patrons très courts (à copier/coller)

```
• ℕ (induction simple)
Base : P(0) (ou P(1)).
H.I. : P(n) vraie.
Pas : P(n) ⇒ P(n+1). Conclusion.
```

• ℕ (induction forte) Base : P(0). H.I. : P(k) vraie pour tous k ≤ n. Pas : {P(k)} ⇒ P(n+1). Conclusion.

• Induction structurelle (mots/listes/arbres) Bases : cas minimaux (ε, nil, feuille/∅). Pas : IH sur sous-objets ⇒ propriété sur l'objet construit.

• Induction bien fondée (mesure m) Base(s) : cas minimal(s). Pas : IH pour tout y avec m(y) < m(x) ⇒ P(x). Conclusion.

Recettes par thème (ultra-concis)

A) Mots (A*)

- Classiques : $|uv| = |u| + |v|$; $\sim(\sim w) = w$; $|\sim w| = |w|$; décomptes $|w|_a$, $|w|_b$.
- Induction structurelle sur w : Base ε ; Pas u·a (on "ajoute 1 lettre").

B) Langage de Dyck

- Équivalence “définition inductive” ⇔ “comptage + tous les préfixes non négatifs”.
- Sens inductif : structurelle sur w = a·u·b·v.
- Sens inverse : bien fondée sur |w| en découpant au premier b qui ferme le a initial.

C) Ordres sur A*

- **Préfixe** : partiel et bien fondé. Définition “inductive” ⇔ “v = u·w”.
- **Lexicographique** (alphabet total) : total mais en général pas bien fondé.

D) Listes

- s(x|y) = s(x)+s(y) ; ℓ(x|y) = ℓ(x)+ℓ(y) ; ℓ(rev x)=ℓ(x) ; rev(rev x)=x.
- Induction structurelle sur la première liste (Base nil ; Pas n|x).

E) Arbres binaires

- Hauteur : h(∅)=0 ; h(⟨g,a,d⟩)=1+max(h(g),h(d))
- Nœuds : n(∅)=0 ; n(⟨g,a,d⟩)=1+n(g)+n(d)
- Feuilles : f(∅)=0 ; f(•)=1 ; sinon f(⟨g,a,d⟩)=f(g)+f(d)
- Borne : $n(t) \le 2^{h(t)} - 1$; (strict) n(t) = 2·f(t) − 1.

F) Relations inductives sur ℕ×ℕ (type Inf1/Inf2)

- Idée : ces règles codent souvent “≤”. Prouver “règles ⇒ ≤” (lecture) et “≤ ⇒ règles” (construction).

Exemple 1 — Mots (langage L sur A={a,b})

Définition de L : (B) b ∈ L. (I) si u∈L et v∈L, alors a·u·v ∈ L.

1. Montrer : pour tout w∈L, $|w|_b = |w|_a + 1$.
2. Montrer (sécurité des préfixes) : pour tout préfixe propre w′ de w, $|w'|_a \ge |w'|_b$.

Solution guidée

(1) Égalité globale — Induction structurelle.

- *Base* : w=b ⇒ |a|=0, |b|=1 ⇒ OK.
- *Pas* : w=a·u·v. IH : $|u|_b=|u|_a+1$ et $|v|_b=|v|_a+1$.
  Alors $|w|_a=1+|u|_a+|v|_a$, et $|w|_b=|u|_b+|v|_b = (|u|_a+1)+(|v|_a+1) = |w|_a+1$.

(2) Préfixes non négatifs — Induction structurelle sur w.

- *Base* : w=b. Préfixe propre seul : ε ⇒ 0≥0.
- *Pas* : w=a·u·v. Un préfixe propre w′ est soit ε, soit “a”, soit “a·x” (x préfixe propre de u), soit “a·u·y” (y préfixe propre de v).

- ε : 0≥0. “a” : 1≥0.
- a·x : $|a| - |b| = 1 + (|x|_a - |x|_b) \geq 1 + 0$.
- a·u·y : $(1 + |u|_a + |y|_a) - (|u|_b + |y|_b) = (|y|_a - |y|_b) \geq 0$ (car IH sur y et (1)).

Exemple : u=b, v=b ⇒ w=a·b·b : |a|=1, |b|=2 et tous les préfixes respectent (2).

Exemple 2 — Dyck (deux sens)

But : “Définition inductive” ⇔ “compte $|\cdot|_a=|\cdot|_b$ et tous les préfixes vérifient $|\cdot|_a \geq |\cdot|_b$”.

Plan court

- **Sens inductif → conditions** (structurelle) : Base ε : OK. Pas w=a·u·b·v : additionner les comptes ; pour les préfixes, séparer en “avant a”, “dans u”, “après b”, et appliquer les IH sur u et v.
- **Conditions → sens inductif** (bien fondée) : si w≠ε, il commence par a et a un premier b qui ferme ce a : écrire w=a·u·b·v ; les conditions se transmettent à u et v (plus courts) ⇒ IH ⇒ reconstruire par la règle inductive.

Exemple 3 — Ordre préfixe : deux définitions équivalentes

D1 (inductive) : ε ≤ v ; si u ≤ v alors a·u ≤ a·v. **D2** (concaténation) : u ≤ v ⇔ ∃w, v = u·w.

Preuve en deux sens (très guidée)

- **D2 ⇒ D1** : si v = u·w, on fait une induction sur la *dérivation* induite par D1 : Base ε ≤ v (prendre w=v). Pas : si a·u ≤ a·v et v = u·w, alors a·v = (a·u)·w ⇒ la règle inductive est satisfaite.
- **D1 ⇒ D2** : récurrence sur |w| tel que v = u·w. Base w=ε ⇒ v=u. Pas w=a·x ⇒ v=u·a·x ; appliquer la règle inductive “remonte” la lettre a, puis IH sur x.

Exemple 4 — Arbres binaires

Propriété : pour tout arbre t, $n(t) \leq 2^{h(t)} - 1$.

Preuve ultra claire

- **Bases** : $t=\varnothing \Rightarrow n=0, h=0 \Rightarrow 0 \le 2^0-1=0$.
- **Pas** : $t=\langle g,a,d\rangle$. IH : $n(g) \le 2^{h(g)}-1$ et $n(d) \le 2^{h(d)}-1$.
  Poser $M=\max(h(g),h(d)) \Rightarrow h(t)=M+1$.
  $n(t)=1+n(g)+n(d) \le 1+(2^{h(g)}-1)+(2^{h(d)}-1) \le 2\cdot 2^M-1 = 2^{M+1}-1 = 2^{h(t)}-1$.

Auto-contrôle express (20 s)

- $P(\cdot)$ bien écrite ? (avec $|\cdot|$, h, n, f, etc.).
- Tous les cas de base traités (ε / nil / $\varnothing$ / feuille) ?
- Pas : j'utilise exactement les constructeurs ($u\cdot a$ / $n|x$ / $\langle g,a,d\rangle$).
- Conclusion écrite clairement ("par induction …").

Résumé en 5 lignes

- Choisis l'induction adaptée ($\mathbb{N}$ / structurelle / bien fondée).
- Écris $P(\cdot)$ avant de commencer.
- Suis la définition : bases puis constructeurs.
- Pour Dyck : "première fermeture" ; pour préfixe : D1 $\Leftrightarrow$ D2 ; pour arbres : h via max, n via addition.
- Conclue toujours par une phrase de fin.